HireAI Copilot

AI-Powered Recruitment Intelligence Platform

Generative AI Engineers

LLM Pipelines, Prompt Engineering & Explainability — 4 Interns

Confidential · Version 1.0 · 6-Week Intern Program

This document is the complete build guide for the Generative AI team. It covers everything four junior interns need to ship the LLM-powered features of HireAI Copilot over a 6-week sprint: resume parsing, the natural-language Copilot query engine, human-readable score explanations, and the shared infrastructure (prompt library, evaluation, caching, cost controls) that makes them all reliable.

IMPORTANT

LLM features are easy to demo and hard to make reliable. Latency, cost, and output consistency are real problems. Cache aggressively, test with adversarial inputs, and never trust an LLM's output without validation.

Table of Contents

1. Team Overview & Scope
2. Tech Stack & Project Setup
3. Individual Intern Allocations
4. Module 1 — Resume Parser (G1)
5. Module 2 — AI Copilot Engine (G2)
6. Module 3 — Explainable AI (G3)
7. Prompt Library Standards
8. Caching, Cost & Rate Limits
9. LLM Evaluation Framework
10. 6-Week Roadmap with Deadlines
11. Cross-Team Integration
12. Definition of Done

01 Team Overview & Scope

1.1 Who We Are

Four junior interns, no team lead. G1, G2, and G3 each own one of the three user-facing GenAI modules end-to-end. G4 owns the shared infrastructure that all three depend on: the prompt library, the evaluation framework, and the caching/cost layer. This split lets the module owners focus on their feature while G4 makes the whole team run reliably and cheaply.

1.2 What We Own

Owner	Module
G1	Resume Parser — converts unstructured resume text into a structured candidate JSON record. Endpoint: /api/parse.
G2	AI Copilot Engine — natural-language queries → ranked candidate results. Endpoint: /api/copilot.
G3	Explainable AI — generates 3-4 sentence explanations of why a candidate scored what they did. Endpoint: /api/explain.
G4	Shared infrastructure — the prompt library (versioned, peer-reviewed), the eval framework (gold sets, nightly runs, regression alerts), the LLM/cache wrapper, cost dashboards, and rate-limit handling. G4 is consumed by G1/G2/G3 every day.

1.3 Scope Boundaries — What We Do NOT Build

- We do not train ML models. ML team owns scoring; we just call /api/score and explain the output.
- We do not ingest or clean CSVs. Data Analysts own that. We read parsed candidates from the database via the web team's APIs.
- We do not build chat UI. W3 (web team) builds the chat interface; we provide the API it calls.
- We do not host LLMs ourselves. We use OpenAI's API (or Gemini if OpenAI quota is tight).

> **NOTE**
> Every LLM call costs money and adds latency. Before reaching for an LLM, ask: can a 5-line Python function do this? If yes, do that. LLMs are for unstructured text understanding, not for tasks regex can solve.

1.4 Working Together

G4 is service-oriented. G1/G2/G3 own product features and depend on G4's infrastructure. Day-to-day collaboration:

- Module owners (G1/G2/G3) write the first version of their prompts and put them in the library. G4 reviews, improves, and standardizes.
- Module owners write their initial eval sets (gold inputs/expected outputs). G4 wires them into the nightly eval runner and posts results.
- All LLM calls go through G4's wrapper (app/core/llm.py). Module owners never call the OpenAI SDK directly — G4 owns retries, caching, model selection, and cost tracking.
- Wednesday 30-minute prompt review session: G4 leads, all four interns attend. Walk through last week's eval deltas, agree on prompt changes, set goals for the coming week.

02 Tech Stack & Project Setup

2.1 Stack

Tool	Why & How
Python 3.11	Standard runtime.
FastAPI	Each module exposes its own service. Same framework as web team for consistency.
OpenAI SDK (openai>=1.0)	Primary LLM provider. Use gpt-4o-mini for parsing/copilot, gpt-4o for explanations (better quality).
LangChain (selectively)	Use ONLY for the Copilot query routing pipeline (G2). Do NOT wrap simple LLM calls in LangChain — adds complexity for no benefit.
Pydantic v2	Schema validation for all LLM JSON outputs.
PyMuPDF (fitz)	PDF text extraction. Faster and more accurate than PyPDF2.
python-docx	DOCX text extraction.
Redis	Cache LLM responses by prompt hash. Critical for latency and cost.
FAISS (faiss-cpu)	Vector search for semantic candidate retrieval (G2).
sentence-transformers	Local embedding model (all-MiniLM-L6-v2). Avoid paying OpenAI for embeddings.
pytest	Unit + evaluation tests.

2.2 Repository Layout

All GenAI code lives in hireai-genai/ as a separate repo from the web team. The web backend proxies requests to us via HTTP. We expose three FastAPI services that can run together or separately.

```
hireai-genai/
├── app/
│   ├── main.py              # FastAPI app, mounts all 3 modules     (G4 owns)
│   ├── core/                                                        # G4 owns
│   │   ├── config.py        # API keys, model selection, env vars
│   │   ├── llm.py           # Single LLM client wrapper — used by all modules
│   │   ├── cache.py         # Redis cache helpers
│   │   ├── eval.py          # Eval framework runner
```

```
│   │   └── cost.py            # Token / spend tracking
│   ├── modules/
│   │   ├── parser/            # G1
│   │   │   ├── routes.py
│   │   │   ├── extract.py     # PDF/DOCX text extraction
│   │   │   ├── prompts.py     # imports from app/prompts/library.yaml (G4)
│   │   │   ├── schemas.py     # Pydantic
│   │   │   └── normalize.py   # skill canonicalization
│   │   ├── copilot/           # G2
│   │   │   ├── routes.py
│   │   │   ├── intent.py      # natural-language → structured query
│   │   │   ├── retriever.py   # FAISS index over candidates
│   │   │   ├── prompts.py
│   │   │   └── pipeline.py    # LangChain pipeline
│   │   └── explain/           # G3
│   │       ├── routes.py
│   │       ├── generator.py
│   │       ├── prompts.py
│   │       └── shap_data.py
│   ├── prompts/               # G4 owns — canonical prompt library
│   │   ├── README.md
│   │   ├── library.yaml       # all prompts, versioned, with changelog
│   │   └── tests/             # eval cases per prompt
│   └── tests/
├── data/
│   ├── eval_sets/             # G4 owns — gold-standard inputs/outputs per module
│   └── eval_results/          # G4 writes nightly run results here
├── scripts/
│   ├── precache_explanations.py   # G4 owns — pre-warm cache before demo
│   └── run_evals.py               # G4 owns — nightly eval runner
├── requirements.txt
└── .env.example
```

2.3 Day-1 Setup

```
# 1. Clone
git clone https://github.com/<org>/hireai-genai.git
cd hireai-genai

# 2. Virtualenv
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# 3. Set up .env (copy from .env.example)
# Required:
#   OPENAI_API_KEY=sk-...
```

```
#   REDIS_URL=redis://localhost:6379
#   WEB_BACKEND_URL=http://localhost:8000/api  # to fetch candidate/job data
#   PARSER_MODEL=gpt-4o-mini
#   EXPLAIN_MODEL=gpt-4o
#   COPILOT_MODEL=gpt-4o-mini

# 4. Run service
uvicorn app.main:app --reload --port 8001

# 5. Verify all 3 endpoints
curl -X POST http://localhost:8001/parse -F "file=@sample.pdf"
curl -X POST http://localhost:8001/copilot -d '{"query":"top python devs"}'
curl -X POST http://localhost:8001/explain -d
'{"candidate_id":"CAND0001","job_id":"JOB0001"}'
```

2.4 The Shared LLM Wrapper

Every module calls LLMs through one wrapper. This gives us central control over caching, retries, model selection, and cost tracking.

```
# app/core/llm.py
import os, json, hashlib, time
from openai import OpenAI
from app.core.cache import cache_get, cache_set

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

def llm_call(prompt: str, model: str = "gpt-4o-mini",
             json_mode: bool = False, temperature: float = 0.2,
             cache: bool = True, ttl: int = 3600) -> str:
    """Single entry point for all LLM calls in this codebase."""
    cache_key = "llm:" + hashlib.sha256(
        f"{model}|{temperature}|{json_mode}|{prompt}".encode()
    ).hexdigest()

    if cache:
        cached = cache_get(cache_key)
        if cached: return cached

    kwargs = dict(
        model=model,
        messages=[{"role": "user", "content": prompt}],
        temperature=temperature,
    )
    if json_mode:
        kwargs["response_format"] = {"type": "json_object"}

    for attempt in range(3):
```

```
        try:
            resp = client.chat.completions.create(**kwargs)
            text = resp.choices[0].message.content
            if cache: cache_set(cache_key, text, ttl=ttl)
            return text
        except Exception as e:
            if attempt == 2: raise
            time.sleep(2 ** attempt)
```

03 Individual Intern Allocations

G1 — Resume Parser

Focus: Convert unstructured resumes (PDF, DOCX, or simulated raw text from candidates.csv) into a validated, structured candidate record.

Core Tasks:

- Build the /parse endpoint: accepts file upload (multipart) OR a candidate_id (uses the row from candidates.csv as 'simulated' raw input).
- Implement text extraction: PyMuPDF for PDFs, python-docx for DOCX. Strip headers/footers, normalize whitespace.
- Design and iterate on the parsing prompt. Register it in G4's prompt library; use JSON mode for guaranteed valid JSON output.
- Define the output Pydantic schema (Candidate). If the LLM returns malformed JSON, retry with a 'your previous output was invalid, here is the error' prompt — max 2 retries.
- Build a skill normalization layer: 'React.js' → 'React', 'Postgres' → 'PostgreSQL', 'JS' → 'JavaScript'. Maintain a YAML mapping file with 100+ canonical entries.
- Build a 50-resume gold eval set (clean + messy inputs). Hand to G4 to wire into the nightly eval runner. Target >90% field accuracy.
- Document the schema for the web team — they'll insert your output into the DB via /api/candidates POST.
- Write the 'why this candidate matches' summary field that appears on candidate cards.
- All LLM calls go through G4's wrapper. Do not import openai directly.

Weekly Milestones:

1. Week 1: PDF/DOCX text extraction works. First parsing prompt registered with G4; returns valid JSON for one sample resume.
2. Week 2: /parse endpoint live with Pydantic validation. Schema retry logic. Tests on 10 sample resumes. First version of eval set handed to G4.
3. Week 3: Skill normalization layer with 50+ canonical mappings. Eval set of 30 resumes; nightly run by G4 reports >85% accuracy.
4. Week 4: Eval set extended to 50 resumes; nightly run reports >90%. Edge cases (no email, two phone numbers, garbled formatting) handled.
5. Week 5: Latency budget verified (<3s per resume). Documentation for web team.
6. Week 6: Final eval run reviewed. Accuracy report written. Demo materials prepared (3 sample resumes for live parse demo).

G2 — AI Copilot Engine

Focus: Natural-language queries from recruiters → structured query → ranked candidates + AI-generated summary.

Core Tasks:

- Build the /copilot endpoint: accepts {query, history?, job_context?} and returns {candidates[], summary, query_interpreted}.
- Build the intent parser: an LLM call that converts a free-text query into a structured filter spec (job_id, skills[], min_experience, status, label, top_k).
- Build a FAISS vector index over all candidates. Embed each candidate's skills + projects + education using sentence-transformers all-MiniLM-L6-v2.
- Re-build the index nightly (or on-demand via /copilot/reindex). The index lives on disk under /data/faiss/.
- Implement the routing logic: simple filter queries ('Python devs with 3 years') hit the database directly via SQL; semantic queries ('candidates similar to Priya Singh') hit FAISS.
- Generate the summary text via a second LLM call: takes the top-K candidates and writes a 2-3 sentence recruiter-friendly summary.
- Maintain the conversation history: accept last 5 turns from frontend, use them to resolve pronouns ('show me his projects').
- Write 5 'flagship' demo queries that work flawlessly. Coordinate with W3 (frontend chat UI) so they pre-load these as suggested chips.
- Build a 30 query-expected-result gold eval set. Hand to G4 to wire into the nightly eval runner.
- Register all prompts in G4's library. All LLM calls go through G4's wrapper.

Weekly Milestones:

7. Week 1: Simple LangChain skeleton. One hardcoded query type works end-to-end ('top N candidates for job X'). First prompt registered with G4.
8. Week 2: Intent parser handles 5 query types (filter by skills, by experience, by job, by label, top-K). First version of eval set handed to G4.
9. Week 3: FAISS index built. Semantic search returns relevant candidates for natural-language descriptions. Nightly evals reporting from G4.
10. Week 4: Summary generation live. 5 flagship demo queries verified working. Conversation history support.
11. Week 5: Eval set of 30 queries; nightly run reports >85% acceptable results. Latency <4s (cache miss); <500ms (cache hit, via G4's wrapper).
12. Week 6: Edge cases polished (typos, ambiguous queries). Demo dress rehearsal queries verified daily.

G3 — Explainable AI

Focus: Generate human-readable, recruiter-friendly explanations for ML-generated scores. No black box.

Core Tasks:

- Build the /explain endpoint: accepts {candidate_id, job_id}, returns {explanation_text, shap_values[], top_strengths[], top_gaps[]}.
- Pull score breakdown from the ML team's /api/score endpoint (skills_match, exp_score, project_score, label).
- Pull candidate skills and job required skills via web team's APIs to compute matched/missing skills lists.
- Design the explanation prompt to produce a structured 3-4 sentence output following a fixed template: (1) overall fit statement, (2) strengths, (3) gaps, (4) experience comment.
- Define the cache key strategy: (candidate_id, job_id, model_version). Use G4's cache wrapper — do not implement Redis directly.
- Get SHAP values from the ML team (M3 generates them). Format them for the frontend: top-5 features with their contributions.
- Write a tone guide: explanations must be neutral, factual, and bias-free. No subjective language ('great candidate'). No mentions of demographics.
- Write an evaluation rubric and 30 gold (input, expected) cases. Hand to G4 to wire into the nightly eval runner. Target >90% pass rate.

Weekly Milestones:

13. Week 1: Sample prompt designed; manual explanations look good for 5 hand-picked test cases.
14. Week 2: /explain endpoint live; calls ML team's /score; produces explanations.
15. Week 3: Cache key strategy implemented through G4's wrapper. SHAP integration spec agreed with ML team (M3).
16. Week 4: SHAP values flowing through; frontend (W1) renders them in candidate drawer.
17. Week 5: Eval rubric scored on 30 samples (run by G4); >90% pass rate. Tone guide enforced.
18. Week 6: Edge cases (perfect-fit candidate, terrible-fit candidate, missing data) all explained gracefully.

G4 — Prompt Engineering, LLM Evaluation & Cost / Cache Infrastructure

Focus: The shared infrastructure that makes G1, G2, and G3 reliable and cheap. You don't ship a feature — you ship the foundation every feature stands on.

Core Tasks:

- Build app/core/llm.py — the single wrapper every LLM call goes through. Owns model selection, retries with exponential backoff, timeout, JSON-mode handling, cost logging, and Redis caching keyed on a normalized hash of (model, temperature, prompt).
- Build app/core/cache.py — Redis helpers. Provide get/set/invalidate functions and a decorator @cached(ttl=...) that module owners can wrap their LLM calls in.
- Build app/core/cost.py — per-call token + dollar tracking. Log every call to a file or Postgres. Generate a daily cost summary and post it in #genai-team Slack each morning.
- Own app/prompts/library.yaml — the single source of truth for every prompt in the system. Each prompt has: id, version, model, temperature, template, changelog. Reject PRs that change prompts without bumping version + changelog.
- Build the prompt review process (Wednesday 30-min sessions): walk through eval deltas, agree on prompt changes, set goals.
- Build app/core/eval.py and scripts/run_evals.py — the nightly eval runner. Reads gold sets from data/eval_sets/, runs each module, scores against expected, writes results to data/eval_results/. Posts a Slack summary every morning: 'Parser: 91%; Copilot: 87%; Explain: 93%. Trend: ↑ on parser, flat elsewhere.'
- Wire each module owner's eval set into the runner: G1's 50-resume parser set, G2's 30-query copilot set, G3's 30-explanation rubric set.
- Detect regressions: if last night's eval drops more than 3 points from the prior week's average, alert in Slack and tag the module owner.
- Build scripts/precache_explanations.py — Week 6 script that warms the cache for all (candidate_id, job_id) pairs that appear in the demo flow, so demo-day calls are zero-latency. Coordinate with G3 on which pairs matter.
- Manage the OpenAI billing dashboard with the PM. Set per-day spend caps. Alert if we're on track to exceed the $200 project budget.
- Build the Gemini fallback: if OpenAI returns 429, automatically retry the same call against Gemini Flash. Test this monthly.
- Onboard each module owner: walk them through how to use the wrapper, register a prompt, register an eval set. Each module owner should be self-sufficient by end of Week 1.

Weekly Milestones:

19. Week 1: app/core/llm.py + app/core/cache.py + app/core/cost.py shipped. Library.yaml structure agreed; first 3 prompts (one per module) committed. Day-1 onboarding session held with G1/G2/G3.
20. Week 2: Eval runner working. G1's parser eval (10 cases) running nightly. Daily cost summary posted in Slack.
21. Week 3: All three module eval sets wired in (50 + 30 + 30 cases). Regression detection live. Library.yaml has 10+ prompts with changelogs.

22. Week 4: Cost tracking dashboards ready for PM. Gemini fallback implemented and tested. Library has 15+ versioned prompts.
23. Week 5: Pre-cache script run for demo flow. All three modules under their latency budgets. Cost on track for <$200 total spend.
24. Week 6: Final eval run reported in #genai-team. Pre-cache re-warmed 24h before demo. Backup plan: cached responses for all demo queries verified working.

04 Module 1 — Resume Parser (G1)

4.1 Output Schema

This is the contract. Web team inserts this exact shape into the candidates table. Once we agree the schema in Week 1, do not change it.

```
# app/modules/parser/schemas.py
from pydantic import BaseModel, EmailStr, Field
from typing import List, Optional

class ParsedCandidate(BaseModel):
    name: str = Field(..., max_length=255)
    email: EmailStr
    phone: Optional[str] = None
    skills: List[str] = Field(default_factory=list)
    experience_years: float = Field(..., ge=0, le=50)
    education: str
    projects: List[str] = Field(default_factory=list)
    raw_text: str        # full original text — kept for audit
    summary: str         # 1-2 sentence "why interesting" snippet
    parse_confidence: float = Field(..., ge=0, le=1)
```

4.2 The Parsing Prompt

Prompt design is critical. The prompt below is the starting point — iterate weekly based on eval results.

```
PARSER_PROMPT = """You are a recruitment data extraction system. Read the resume
below
and extract structured fields. Return ONLY valid JSON matching this exact schema:

{{
  "name": "string",
  "email": "string",
  "phone": "string or null",
  "skills": ["array of strings, technical skills only, no soft skills"],
  "experience_years": <number, total years of professional experience>,
  "education": "highest degree + institution, e.g. 'B.Tech, IIT Madras'",
  "projects": ["array of project titles, max 5"],
  "summary": "1-2 sentences on what this candidate's strengths are",
  "parse_confidence": <float 0-1, how confident you are the resume is well-formed>
}}

Rules:
```

```
- If experience years are not explicitly stated, infer from earliest job date.
- Normalize skills: 'React.js' → 'React', 'Postgres' → 'PostgreSQL'.
- DO NOT INCLUDE soft skills like 'teamwork' or 'leadership' in the skills array.
- If the email is missing or malformed, return parse_confidence < 0.5.
- Output JSON only. No markdown, no explanation.

Resume:
```
{resume_text}
```
"""
```

4.3 Skill Normalization

LLMs are inconsistent in how they output skill names. Normalize after parsing using a YAML map maintained in the repo.

```
# app/modules/parser/skill_map.yaml
React: ["React.js", "ReactJS", "React JS", "react"]
PostgreSQL: ["Postgres", "postgres", "PSQL"]
JavaScript: ["JS", "Javascript", "ECMAScript"]
TypeScript: ["TS", "Typescript"]
Python: ["python", "Python3", "Py"]
Node.js: ["NodeJS", "Node JS", "node"]
# ... target: 100+ entries by Week 3
```

4.4 Acceptance Criteria

- ✓ 90% field extraction accuracy on the 50-resume eval set.
- ✓ Skill normalization map has 100+ entries.
- ✓ Average parse latency < 3 seconds per resume.
- ✓ Pydantic validation rejects all malformed outputs; retry logic recovers gracefully.
- ✓ Endpoint passes 5 adversarial inputs (no email, multiple phones, non-English text, scanned PDF, blank file).

05 Module 2 — AI Copilot Engine (G2)

5.1 How the Copilot Pipeline Works

25. Receive {query, history, job_context} from the web team's proxy.
26. Intent classification (LLM call, JSON mode): is this a 'filter' query or a 'semantic' query?
27. If filter: extract structured fields (skills, min_experience, job_id, label, top_k) and run a SQL query.
28. If semantic: embed the query, search FAISS for top-K candidate IDs, then fetch full records via web team's API.
29. Re-rank candidates by composite score (already in DB).
30. LLM call #2: generate a 2-3 sentence summary describing the result set.
31. Return {candidates[], summary, query_interpreted}.

5.2 Intent Parser Prompt

```
INTENT_PROMPT = """Convert this recruiter query into a structured search.
Return ONLY a JSON object with this schema:
{{
  "type": "filter" | "semantic",
  "job_id": "string or null",
  "skills_required": ["array of skill names"],
  "min_experience": <number or null>,
  "label_filter": "Good Fit" | "Average Fit" | "Poor Fit" | null,
  "status_filter": "Applied" | "Shortlisted" | "Interviewed" | "Hired" | "Rejected"
| null,
  "top_k": <integer, default 10>,
  "free_text": "<the query without the filters extracted, if anything remains>"
}}

Examples:
Query: "Show top 5 Python developers for JOB0012"
Output: {{"type":"filter","job_id":"JOB0012","skills_required":["Python"],
         "min_experience":null,"label_filter":null,"status_filter":null,
         "top_k":5,"free_text":""}}

Query: "Candidates similar to Priya who built fintech apps"
Output: {{"type":"semantic","job_id":null,"skills_required":[],
         "min_experience":null,"label_filter":null,"status_filter":null,
         "top_k":10,"free_text":"similar to Priya, built fintech apps"}}

Query: "{query}"
```

```
Output:"""
```

5.3 Five Flagship Demo Queries

These five queries MUST work flawlessly by Week 5. Coordinate with W3 to pre-load them as suggested chips. Test them end-to-end every single day in Week 5 and 6.

32. Show top 5 Python developers for JOB0012
33. Who has React and Node.js with 2+ years of experience?
34. Compare top 3 candidates for the ML Engineer role
35. List all Good Fit candidates for JOB0003
36. Which roles have the biggest skill gap right now?

5.4 FAISS Index Build

```
# app/modules/copilot/retriever.py
import faiss, numpy as np, pickle, requests
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")

def build_index():
    candidates = requests.get(f"{WEB_BACKEND_URL}/candidates?limit=2000").json()
    texts = [
        f"{c['name']}. Skills: {', '.join(c['skills'])}. "
        f"Education: {c['education']}. "
        f"Projects: {', '.join(c['projects'])}."
        for c in candidates
    ]
    embeddings = model.encode(texts, normalize_embeddings=True)
    index = faiss.IndexFlatIP(embeddings.shape[1])
    index.add(embeddings.astype("float32"))
    faiss.write_index(index, "data/faiss/candidates.index")
    with open("data/faiss/ids.pkl", "wb") as f:
        pickle.dump([c["id"] for c in candidates], f)
```

5.5 Acceptance Criteria

- ✓ All 5 flagship demo queries return correct, demo-quality results.
- ✓ Eval set of 30 queries: >85% return acceptable top-K results.
- ✓ Average response time <4 seconds (cache miss); <500ms (cache hit).

- ✓ Conversation history: pronoun resolution works for 'show me his/her projects' style follow-ups.
- ✓ Graceful failure: if intent parsing fails, return a friendly 'I didn't understand — try rephrasing' message instead of an error.

06 Module 3 — Explainable AI (G3)

6.1 What This Module Produces

For every candidate-job pair, we generate a 3-4 sentence explanation that a recruiter can read in 10 seconds and understand WHY this candidate was scored the way they were. This is what makes our product trustworthy and not a black box.

6.2 Output Schema

```
# app/modules/explain/schemas.py
from pydantic import BaseModel
from typing import List, Tuple

class Explanation(BaseModel):
    candidate_id: str
    job_id: str
    explanation_text: str          # 3-4 sentences
    top_strengths: List[str]       # e.g. ["Python (5y exp)", "Spark", "AWS"]
    top_gaps: List[str]            # e.g. ["Kafka", "Airflow"]
    shap_values: List[Tuple[str, float]]  # [("skills_match", 0.42), ...]
    model_version: str
    generated_at: str
```

6.3 The Explanation Prompt

Strict template. Bias-free. Factual. No subjective adjectives.

```
EXPLAIN_PROMPT = """You are explaining an AI-generated candidate score to a
recruiter.
Be concise, factual, and neutral. Do NOT use subjective words like 'great',
'excellent',
'unfortunately', or 'sadly'. Do NOT mention demographics, age, gender, or location.

Inputs:
- Candidate name: {name}
- Job title: {job_title}
- Composite score: {score}/100, label: {label}
- Skills match: {skills_match}/100  ({matched_count} of {required_count} required
skills)
- Experience: {candidate_exp} years (job requires {required_exp})
- Project relevance: {project_score}/100
- Matched skills: {matched_skills}
```

```
- Missing skills: {missing_skills}

Write exactly 3-4 sentences in this structure:
1. State the candidate name, job title, score, and label.
2. State strengths: which required skills they match and their experience.
3. State gaps: which required skills are missing.
4. Optional: comment on project relevance if score >75 or <40.

Output the explanation only — no preamble."""
```

6.4 Caching Strategy

Explanations are deterministic given (candidate_id, job_id, ml_model_version). Once generated, never regenerate unless the model version changes. Cache key:

```
cache_key = f"explain:{candidate_id}:{job_id}:v{model_version}"
# TTL: 30 days. Manually invalidate on model version bump.
```

> **RULE**
>
> Pre-generate explanations for the demo: at the start of Week 6, run a script that warms the cache for all (candidate_id, job_id) pairs that appear in the demo flow. This guarantees zero LLM latency during the live demo.

6.5 Acceptance Criteria

- ✓ 30-sample eval rubric scores >90% on factual accuracy + tone neutrality.
- ✓ Cache hit on repeated calls: 100%.
- ✓ Cache miss latency <3 seconds.
- ✓ Edge cases: handles candidates with 0 missing skills, candidates with all missing skills, missing project data, and missing experience info.
- ✓ SHAP values for top-5 features integrated and rendered in frontend candidate drawer.

07 Prompt Library Standards

All prompts live in app/prompts/library.yaml. Every prompt is versioned. When you change a prompt, bump its version, run the eval, and document the change.

7.1 Prompt File Format

```
# app/prompts/library.yaml
parser_main:
  version: 3
  description: "Main resume parsing prompt with JSON output."
  model: gpt-4o-mini
  temperature: 0.1
  template: |
    You are a recruitment data extraction system...
    ...
    Resume:
    ```
    {resume_text}
    ```
  changelog:
    - v1: initial
    - v2: added skill normalization rules
    - v3: lowered temperature, fixed soft-skill leakage

intent_classifier:
  version: 2
  ...

explanation:
  version: 4
  ...
```

7.2 Prompt Review Process

37. Author drafts a new prompt version in a feature branch.
38. Author runs the eval set (via G4's runner) against both old and new prompts.
39. Author opens a PR with: prompt diff, eval delta (e.g. accuracy: 87% → 91%), and 3 sample inputs/outputs.
40. G4 reviews and approves before merge — this guarantees consistent style, naming, and metadata across the library.
41. On merge, version is bumped in library.yaml and a one-line entry added to the changelog.

7.3 Anti-Patterns to Avoid

- Don't chain 5 LLM calls when 1 will do. Each call adds 1-2 seconds and a chance of failure.
- Don't include the entire user query in the cache key without normalization. 'top python devs' and 'Top Python Devs' should hit the same cache.
- Don't trust LLM JSON output without Pydantic validation, even with JSON mode enabled.
- Don't include real PII or proprietary data in your prompts during testing — it gets logged in OpenAI's systems.

08 Caching, Cost & Rate Limits

This entire section is owned by G4. Module owners just call llm_call(...) — G4's wrapper handles caching, retries, model selection, and cost tracking transparently.

8.1 Cost Targets

Total OpenAI spend across the 6-week project must stay under $200. G4 monitors daily and posts a Slack summary every morning. PM has access to the billing dashboard and will flag overages weekly.

Module	Model	Calls/day budget	Strategy (G4 enforces)
Parser (G1)	gpt-4o-mini	~200	Cache by file hash. Most resumes parsed once.
Copilot (G2)	gpt-4o-mini	~500	Cache by normalized query. Reuse intent results.
Explain (G3)	gpt-4o	~1000 (one-time)	Pre-generate cache for top 1000 (candidate, job) pairs in Week 4.

8.2 Pre-Cache Script (G4 owns, runs weekly)

```
# scripts/precache_explanations.py — owned by G4
import requests
from app.modules.explain.generator import generate_explanation

apps = requests.get(f"{WEB_BACKEND_URL}/applications?limit=2000").json()
for a in apps:
    try:
        generate_explanation(a["candidate_id"], a["job_id"])
    except Exception as e:
        print(f"FAIL {a['candidate_id']}/{a['job_id']}: {e}")
print(f"Cached {len(apps)} explanations.")
```

8.3 Rate Limit Handling (G4 owns)

OpenAI has tier-based rate limits. If we hit them mid-demo, we look bad. G4's wrapper handles all of this transparently:

- Exponential backoff retry (built into the llm_call wrapper).
- Pre-cache aggressively before the demo (G4 runs the script in Week 5 and again 24h before demo).
- Gemini fallback: if OpenAI returns 429, the wrapper automatically swaps to Gemini Flash for that single call. G4 tests this monthly.
- Per-minute and per-day spend caps configured in the OpenAI dashboard by G4.

09 LLM Evaluation Framework

LLM outputs are non-deterministic. The only way to know if a prompt change is an improvement is a frozen eval set with measurable metrics. Each module owner defines and maintains their gold eval cases; G4 wires them into the nightly runner and reports results.

9.1 Eval Set Structure

Module	Set Size	Metric	Target
Parser (G1 → G4)	50 resumes	Field-level accuracy (precision per field)	>90% across all fields
Copilot (G2 → G4)	30 queries	Top-K result match against gold list (Recall@K)	>85% R@5
Explain (G3 → G4)	30 (cand, job) pairs	Human rubric: factual / clear / unbiased (1-3 each)	Avg ≥ 2.5 / 3 across all dims

9.2 Eval Runner (G4 owns)

```
# app/core/eval.py — runs nightly via cron, owned by G4
import json
from pathlib import Path

def run_eval(module: str):
    eval_set = json.loads(Path(f"data/eval_sets/{module}.json").read_text())
    results = []
    for case in eval_set:
        actual = run_module(module, case["input"])
        score = score_case(case["expected"], actual)
        results.append({"id": case["id"], "score": score})
    accuracy = sum(r["score"] for r in results) / len(results)
    print(f"{module} eval: {accuracy:.1%}")
    Path(f"data/eval_results/{module}_latest.json").write_text(
        json.dumps({"accuracy": accuracy, "results": results}, indent=2))
```

9.3 Reporting

Every morning, G4 posts the previous night's eval summary in #genai-team Slack: 'Parser: 91% (↑2 vs last week); Copilot: 87% (flat); Explain: 93% (↑1).' Module owners are responsible for investigating any regressions in their module.

10 6-Week Roadmap with Deadlines

IMPORTANT

LLM features look magical until they break in front of a stakeholder. Build evaluation discipline from Week 1 — accuracy you can't measure is accuracy you can't trust.

Week 1 — Foundations

Intern	Friday EOD Deliverable
G1	PDF/DOCX text extraction works. First parsing prompt registered with G4; returns valid JSON for one sample resume.
G2	FastAPI scaffold; LangChain skeleton; one hardcoded query type works end-to-end.
G3	Sample explanation prompt designed; manual outputs reviewed for 5 hand-picked cases.
G4	llm.py + cache.py + cost.py shipped. Library.yaml structure agreed; first 3 prompts committed. Onboarding session held with G1/G2/G3.

Week 2 — Modules Live

Intern	Friday EOD Deliverable
G1	/parse endpoint live with Pydantic validation and retry logic. Tested on 10 sample resumes. First eval set handed to G4.
G2	Intent parser handles 5 query types. /copilot endpoint returns mock-but-shaped results.
G3	/explain endpoint live; calls ML team's /score; produces explanations for sample inputs.
G4	Eval runner working. Parser eval (10 cases) running nightly. Daily cost summary posted in Slack.

Week 3 — Quality & Caching

Intern	Friday EOD Deliverable
G1	Skill normalization map with 50+ entries. Eval set of 30 resumes; nightly run reports >85%

Intern	Friday EOD Deliverable
	accuracy.
G2	FAISS index built; semantic search returns relevant candidates.
G3	Cache key strategy implemented through G4's wrapper. SHAP integration agreed with ML team.
G4	All three module eval sets wired in. Regression detection live. Library.yaml has 10+ prompts with changelogs.

Week 4 — Polish & Integration

Intern	Friday EOD Deliverable
G1	Eval set extended to 50 resumes; nightly run reports >90%. Edge cases handled.
G2	Summary generation live. 5 flagship demo queries verified. Conversation history support.
G3	SHAP values flowing through; web team renders them in candidate drawer.
G4	Cost tracking dashboards ready for PM. Gemini fallback implemented. Library has 15+ versioned prompts.

Week 5 — Demo Prep

Intern	Friday EOD Deliverable
G1	Latency <3s/resume verified. Documentation for web team.
G2	Nightly eval reports >85% acceptable on 30 queries. Latency <4s (cache miss); <500ms (cache hit).
G3	Eval rubric on 30 samples >90% pass rate. Tone guide enforced.
G4	Pre-cache script run for demo flow. All three modules under their latency budgets. Cost on track for <$200 total spend.

Week 6 — Final

Intern	Friday EOD Deliverable
G1	Final eval run reviewed. Accuracy report written. 3 demo resumes prepared for live demo.
G2	Edge cases polished. Demo dress rehearsal — all 5 flagship queries verified daily.
G3	All edge cases (perfect, terrible, missing data) handled gracefully.
G4	Final eval run reported. Pre-cache re-warmed 24h before demo. Backup cached responses for all demo queries verified.

11 Cross-Team Integration

11.1 Who We Wait On

From	What	When
Web (W4, W5)	Working /api/candidates and /api/jobs to fetch context	End of Week 1
Web (W6)	Proxy endpoints to call our services	Week 4
ML (M4)	Working /api/score endpoint	End of Week 3
ML (M3)	SHAP feature importance values per (cand, job)	Week 4
Data (DA1)	Cleaned candidates loaded in DB	Week 1

11.2 Who Waits on Us

Who	What
Web (W3)	Working /copilot endpoint by Week 4 — chat UI integration
Web (W1)	Working /explain endpoint by Week 4 — candidate drawer integration
Web (W6)	Schema for all three endpoints by end of Week 1 so they can build proxies

11.3 The 'Mock Contract' Discipline

By end of Week 1, all three GenAI endpoints return a hardcoded sample response in the EXACT JSON shape that real responses will have. This unblocks the web team to build UI and the ML team to plan integration. The web team builds against our mocks; we replace mocks with real logic gradually.

12 Definition of Done

A GenAI feature is 'done' only when ALL of these are true:

- ✓ Endpoint deployed and reachable from the web backend.
- ✓ Pydantic validation on all inputs and outputs.
- ✓ Redis caching active where applicable, with documented TTL.
- ✓ Eval set defined and current accuracy reported in repo.
- ✓ Latency budget met (p95 measured and documented).
- ✓ Code merged to main with at least one peer review from another GenAI intern.
- ✓ Prompts versioned in library.yaml with a changelog.
- ✓ Demo-able live in under 3 minutes without script.

> **CRITICAL**
> If your feature can fail unpredictably (LLM hallucinations, timeouts), build the failure path before you build the success path. The recruiter must NEVER see a stack trace.

HireAI Copilot — Generative AI Engineers Documentation

Built by 4 Junior Interns. Guided by evals. Powered by careful prompts.

Confidential · Version 1.0 · 6-Week Sprint